3 - Hybrid System Modeling and Simulation

Freitag, 4. Juli 2025 15:06

Datum	**06.07.2025 bis 07.07.2025**
Kategorie	**Tool Analysis / Framework Inspiration**
Quelle	**Principles of Object-Oriented Modelling and Simulation with Modelica**
Arbeitspaket	AP4: Analyse bestehender Tools

Notes:

Part 3

7 Hybrid System Specification

7.1 Introduction

- State of hybrid models is described using continuous-time and discrete-time variables
- Continuous change in the value of the continuous-time state variables
- Instantaneous changes in the total state -> events
- Algorithm is devised to switch between the solution of the continuous-time problem, and the execution of the events

7.2 The OHM formalism

- Omola Hybrid Model
- Model can be described by the tuple: $M = (q, x, y, E, G, H, \Phi, \Delta)$
- $q = (q_1, \dots, q_n)$: vector that contains the model discrete-time variables (real, integer, Boolean, string type)
- $x = (x_1, \dots, x_n)$: vector that contains the continuous-time state variables (real)
- $y = (y_1, \dots, y_n)$: vector that contains the continuous-time algebraic variables
- E is a set that contains all the possible types of events
- G is the set of expressions that define the continuous-time behavior pf the model
- H is a set of Boolean expressions, named invariant expressions
    - Set of admissible states: make the value of every Boolean expression to be true
    - Set of non-admissible state: make the value of at least one Boolean expression to be false
    - Invariant expression describe trigger conditions of events
- $\Phi: H \rightarrow E$ is a function that associates an event type to each invariant expression
- Δ is a set of vector expressions that describe the instantaneous change in the model variables produced by the execution of each event type
    - At event execution time t_e a discontinuous change in the model variables takes place
    - Previous values change to the new values
    - Vector equation associated to the event, together with all the continuous-time equations of the model are solved jointly to calculate the value of the variables at the event

7.3 Model Specification and simulation algorithm

1. Solution of the continuous-time problem
    a. Is described by the DAE system
    b. Implies solving non-linear system of simultaneous equations and performing numerical integration
    c. Discrete-time variables have constant known values during the solution the continuous-time problem
    d. Values of discrete-time variables only change when executiong the events

2. Detection of events
    a. Carried out by checking the invariant expressions during the simulation of the continuous-time problem
    b. Numerical solution of the continuous-time problem is stopped when an invariant expression changes from true to false
    c. Iterative algorithm for finding the trigger time of the event is started
3. Determination of the event trigger time
    a. Numerical integration of the DAE system advances in time steps
    b. Event can be detected at a time later than ist trigger time
    c. When event is detected, an iterative method is employed to locate the event trigger time within the last integration step
    d. Time interval is obtained, satisfying that the interval contains the event trigger time and the interval length is below a certain tolerance
    e. It is assumed that the event trigger time is the right limit of the interval and is named te
4. Execution of the event
    a. New variable values calculated at the event time must be consistent initial values for the continuous-time problem
    b. Is resumed after executing the event
    c. New values must satisfy all the equations that describe the continuous-time behavior of the model
    d. Event execution is also refereed to as solving the restart problem

7.4 Model Specification and Modelica description

- Events are described using when clauses, and if sentences and clauses
    - Allow to describe changes in the value of discrete-time variable
    - Reinitialize the value of continuous time variables
- When clauses composed of
    - Logical expression: describing the trigger condition
    - Set of equation: instantaneous equations
        - New values of the variables appear explicitly indicated

```
when logical expression then
  instantaneous equations
end when;
```

- Equations describing changes in the value of discrete-time variables have to be written as assignments
    - New value assigned to the left hand side variable is calculated evaluating the expression on the right hand side
- Value of continuous-time state variables is reinitialized in Modelica using the reinit function with 2 arguments
    - State variable to be reinitialized
    - Expression employed for calculating the new value

7.5 Models with variable structure

- Variable structure: mathematical description can change during simulation rum
- Models of this type can be in different modes
- Each mode is described by a particular system of equations
- During simulation run, transitions between modes are taking place according to predefined conditions, producing the corresponding changes in the model mathematical description
- Discrete event dummy variable in combination with if sentence
- Computational causality of the switch's constitutive relationship depends on the value of the control variable
    - Can change during the simulation run
    - Assigning the computational causality of the complete model, the switch's constitutive relationship will be part of an algebraic loop
- The number of DoF may depend on the switch mode
- Modelica environment does not support sthe simulation of models with variable number of

DOF
    - Model developer needs to modify the modeling hypotheses in order to avoid runtime changes in the number of DoF
- Resistive switch
    - Difficulties associated to the use of ideal switches are avoided using resistive switches
    - Constitutive relationships are algebraic equations that contain the connector pressures and the volumetric flow rate

7.6 Model Initialization

- Calculation of the model variables at the initial time
- If the model contains nonlinear algebraic loops, iterative methods are applied using as initial guess the corresponding values provided by the model developer
- Initial guess for iterating the algebraic loop is the value of the variable calculated at the previous time step
- Reduction of the integration time step length if the iterative method does not converge
- Problematic when an event produces instantaneous changes in the model state
    - Abrupt change in the model state can make the actual solution of the algebraic loop to be too far from the initial guess
    - Iteration of the restart problem does not converge
- Initialization problem consists in calculating consisten values for all the model variables at the initial time
- Unknown variables are calculated by solving the equations and algorithms that describe the continuous-time behavior of the model, and a additional constraints (initial conditions)

8 Event Detection and Handling

8.1 Introduction

- Detection and handling of events
- Concept of the crossing function to detect events
- Chattering

8.2 Simultaneous events

- execution of an event can generate the triggering of another event in that same time instant
- happens when the solution of the restart problem does not satisfy one of the invariants
- event corresponding to this invariant is immediately executed
- several events can be sequentially executed until all invariants are satisfied
- solution of the continuous-time problem is resumed afterwards
- event chain: execution of a sequence of events
- Challenges:
    - several events can be detected simultaneously during the solution of the continuous-time problem
    - several invariants can be not satisfied at certain step in the execution of an event chain
    - necessary to establish a criterion to decide how to execute these simultaneously triggered events
- Execution order may be relevant when the events affect parts of the model that interact among each other
- Several methods, deterministic and stochastic, to decide the execution order of the events triggered simultaneously

Example:

Suppose that the events e1 and e2 have been triggered simultaneously.
The event e1 is executed first, because it has been defined before the event e2.
The solution of the restart problem satisfies one of the three following conditions:

1. If all the invariant expressions are satisfied, then the solution of the continuous time problem is resumed
2. If only one invariant expression is not satisfied, the event associated to this invariant is executed
3. If several invariant expressions are not satisfied, the event with less order of definition, among the events associated to these invariant, is executed

Algorithm 1:

1. Execute the event with less order of definition among the triggered events. An event is triggered when the value of its invariant expression is false
2. Check whether there are triggered events. If this is the case, go to step 1. Otherwise, resume the solution of the continuous-time problem.

Algorithm 2:

1. Determine and sort out, according to the definition order, the set of triggered events. The set is named E'.
2. If the set E' is empty, resume the solution of the continuous-time problem, finishing this algorithm.
3. Execute the first event of the E' set.
4. Consider the next event of the sorted set E'. If this event has not been executed yet and is still triggered, this event is executed.
5. If every event of E' has been examined, go to Step 1. Otherwise go to step 4.

- Other approach: executing simultaneously all the triggered events
- Single-assignment rule
    - all the instantaneous changes in a continuous-time or discrete-time state variable must be described in a single instantaneous equation
    - guarantees that the same state variable is not changed by two instantaneous equations simulatneous active
    - eliminates the risk of executing simultaneously several events that assign different values to a same state variable
- Event logging: writing information on the executed events in the message window during the simulation

8.3 Crossing function

- modeling environments of hybrid systems typically use crossing-functions for detecting events
- event conditions are automatically translated into crossing functions
    - are watched during the continuous-time problem solution
- is an expression whose result is
    - positive while the event condition is true
    - negative while the event condition is false
- function crosses the zero value at the time instant in which the event condition changes from true to false or vice versa
- Change of the event condition from false to true is detected when z crosses eveps with positive slope
- Change of the event condition from true to false when z crosses -eveps with negative slope
- if z crossing function initially remains inside the interval (-eveps, eveps) due to the initial conditions, the value of the crossing function is assumed to be zero
- this mechanism may result in numerical artifacts that condition the simulation result

8.4 Determination of the event instant

- depending on its trigger condition, events can be classified into time events and state events
    - time events can be classified into exogeneous and endogeneous
        - exogeneous: trigger time is specified in the model
        - endogeneous: time is computed during the simulation execution as a result of the execution of a previous time event or state event
        - time step of the integration algorithm is modified so that the evaluation time is equal to the time event
    - state events are triggered when the system satidfies certain conditions
        - trigger time of state events is not known in advance
        - must be calculated during the simulation (event iteration)
- Two branch equation
    - crossing functions for detecting state events
    - when a state event is detected, the integration is halted, and the event iteration is started

- event iteration: iterative algorithm to determine the time instant in which the event is triggered
- calculation implied the evaluation of the equation
- extending the old branch beyond its validity domain
- old branch is switched to the new time once the event trigger time is determined
- restart problem is solved using the new branch of the equation
- integration algorithm is resumed, starting at the event instant, using the new branch of the function
- event detection procedure requires evaluating equation branches beyond their definition domain
  - runtime numerical error if this is not possible
  - trajectory in the state space crosses the definition domains of the equation branches
- noEvent() operator
  - in case of a change of branch, modeling environment must not iterate to find the precise time instant in which the vent was triggered
  - avoids event iteration: performing a textual handling of the equation
- textual handling implies to integrate across the branch switching
  - discontinuity between the branches: integration algorithm may fail
  - integration algorithms are designed on the assumption that the function to integrate and its derivatives are continuous

8.5 Chattering

- Simulation exhibits chattering if the number of state events executed during the simulation is large in comparison with the number of integration steps
- if state event is detected, trigger time is calculated and the restart problem is solved
- chattering significantly slows down the simulation
- noEvent() operator may avoid chattering
- Only way to avoid chattering is to modify the modeling hypotheses
- Allows user to set the types of variables to store in file and also at which time instants
  - results are stored at equidistant time instants and at event instants

9 Hybrid Modeling Practice

9.1 Introduction

- hybrid models combine continuous time behavior with events
- event is a set of actions that are triggered when a certain condition is satisfied

Actions that Modelica allows to perform in an event:

- Change in the model structure
  - event can generate a change in the mathematical structure of the model
  - change in the equations that describe the model behavior
- update the value of a discrete-time variables
  - modify the value of one or more discrete-time variables
  - value of a discrete -time variable is constant between two consecutive events
  - changing only at event instants
- Reinitialization of continuous-time state variables
  - change the value of a continuous-time variable
  - variable whose value is reiitialized in the event action has to be a state variable

9.1.1 If sentence and clause

- allow to describe models with a variable structure
- both can be included in equation and algorithm sections
- if sentence allows to describe functions with several branches

$$expr_1 = \textbf{if}\ cond\ \textbf{then}\ expr_2\ \textbf{else}\ expr_3;$$

$$var := \textbf{if}\ cond\ \textbf{then}\ expr_1\ \textbf{else}\ expr_2;$$

- Else branches:

$expr_1 =$ **if** $cond_1$ **then** $expr_2$
elseif $cond_2$ **then** $expr_3$ **else** $expr_4$;

$var :=$ **if** $cond_1$ **then** $expr_1$
elseif $cond_2$ **then** $expr_2$ **else** $expr_3$;

- If clause:

if *cond* **then**
equations
else
equations
end if;

9.1.2 Textual Handling of if expressions

- Modelica modeling environments perform an event-bases handlig of if expressions
- noEvent() function in the logical condition of an if expression indicates that the if expression has to e handled textually

$expr_1 =$ **if noEvent**$(cond)$ **then** $expr_2$ **else** $expr_3$;

$var :=$ **if noEvent**$(cond)$ **then** $expr_1$ **else** $expr_2$;

  - evaluates boolean condition to chose the branch
  - then computes the corresponding variable using the chosen branch
- allows to avoid runtime numerical errors when the branches cannot be extended beyond its validity range

9.1.3 When clause

when *cond* **then** instantaneous equations **end when**;	**when** $\{cond_1, \ldots, cond_n\}$ **then** instantaneous equations **end when**;

- cond is a Boolean expression or vector of boolean expressions
- when clause is triggered, each time any of the vector component changes its value from false to true

Types of instantaneous equatins:

- Difference equations
  - describing how the new values of discrete-time variables are evaluated
  - depending on wheter the when clause is included inside an equation or algorithm section
- reinit sentence
  - change abruptly the value of continuous-time state variables
  - first argument must be a state variable

Built-in functions for Boolean condition:

- Sample() function triggers periodically the when clause
- initial() function triggers the when clause at the model initialization
- terminal() function triggers when the ending condition of the simulation is satisfied

- LogVariable() function writes the actual value of the variable to the message window